

łańcuch znaków

Zbigniew Marszalek

Klasa String

W języku Java zdefiniowana została klasa String przeznaczana do manipulacji na ciągach znakowych. W celu utworzenia egzemplarza klasy String i przypisania mu jakiejś wartości użyjemy następującej instrukcji:

```
String str1 = "Łańcuch znaków";
```

Do deklaracji łańcucha znaków str1 używamy słowa kluczowego String i przy posługiwaniu się łańcuchami znakowymi należy zwracać uwagę na następującą własność tej klasy: łańcuch jest ciągiem znaków typu char.

Łańcuchy są niezmiennie (ang. *immutable*). Oznacza to, że po utworzeniu łańcucha nie można go zmienić w ramach tej samej lokalizacji w pamięci. Wszystkie dokonywane modyfikacje łańcucha (zamiana znaków, usuwanie znaków albo dopisywanie) powoduje przepisanie łańcucha w inne miejsce w pamięci operacyjnej.

Ze względu na to, że klasa String jest klasą referencyjną łańcuchy znakowe są porównywane według wartości, a nie miejsca składowania w pamięci operacyjnej. Do obsługi łańcuchów znakowych przeznaczona są klasy String, StringBuffer i StringBuilder.

Klasy String, StringBuffer i StringBuilder są zdefiniowane w pakiecie java.lang. Z tego powodu są dopisywane automatycznie w każdym pakiecie Java. Deklaracje wszystkich tych klas zawierają modyfikator *final*, co oznacza, że nie można po nich dziedziczyć. Takie rozwiązanie umożliwia przeprowadzenie optymalizacji zwiększającej wydajność dla typowych operacji na łańcuchach znakowych. Wszystkie trzy klasy implementują interfejs CharSequence.

Konstruktor klasy String

Klasa String ma kilka konstruktorów. W celu utworzenia pustego obiektu wystarczy użyć konstruktora domyślnego np..

```
String s = new String();
```

Aby utworzyć obiekt String zainicjowany przez tablicę znaków można użyć konstruktora:

String(**char** *chars*[])

```
char znaki[] = { 'x', 'y', 'z' };  
String s = new String(znaki);
```

Kolejny konstruktor umożliwia konstrukcje łańcucha z ograniczonej liczby znaków

`String( char chars[], int startindex, int numChars)`

```
char znaki[] = { 'u', 'w', 'x', 'y', 'z' };  
String s = new String(znaki);
```

Konstruuje obiekt *String* zawierający znaki x, y i z.

Aby utworzyć obiekt zawierający te same znaki, co inny obiekt *String* korzystamy z konstruktora:

`String( String strObj)`

```
char c[] = { 'J', 'a', 'v', 'a' };  
String s1 = new String(c);  
String s2 = new String(s1);
```

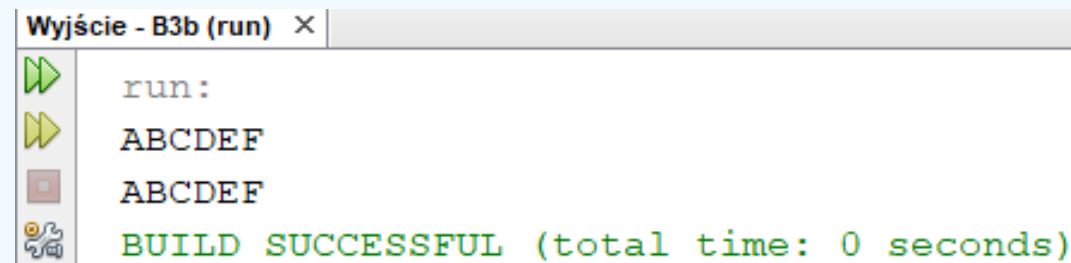
Choć typ `char` w języku Java jest 16 – bitowy, gdyż reprezentuje znaki w standardzie Unicode, to do utworzenia łańcucha znaków możemy użyć 8 – bitowych kodów ASCII za pomocą konstruktorów:

`String( byte asciiChars[] )`

`String( byte asciiChars[], int startIndex, int numChars )`

```
byte ascii[] = { 65, 66, 67, 68, 69, 70 };  
String s3 = new String(ascii);  
String s4 = new String(s3);
```

```
System.out.println(s3);  
System.out.println(s4);
```



```
Wyjście - B3b (run) ×  
run:  
ABCDEF  
ABCDEF  
BUILD SUCCESSFUL (total time: 0 seconds)
```

UWAGA: Istnieją rozszerzone wersje konstruktorów konwersji z bajtów na znaki, w których można określić sposób kodowania użytego przy konwersji.

Długość ciągu znaków

Długość ciągu znaków to liczba zawartych w nim znaków. Do uzyskania długości łańcucha służy metoda *length()*

int length()

```
char c[] = { 'J', 'a', 'v', 'a' };  
String s1 = new String(c);  
System.out.println(s1.length());
```

Literały tekstowe

Istnieje łatwiejszy sposób utworzenia egzemplarza klasy String. Wystarczy wpisać literał tekstowy i Java utworzy obiekt String.

```
String s6 = "wxyz";  
System.out.println("wxyz".length());
```

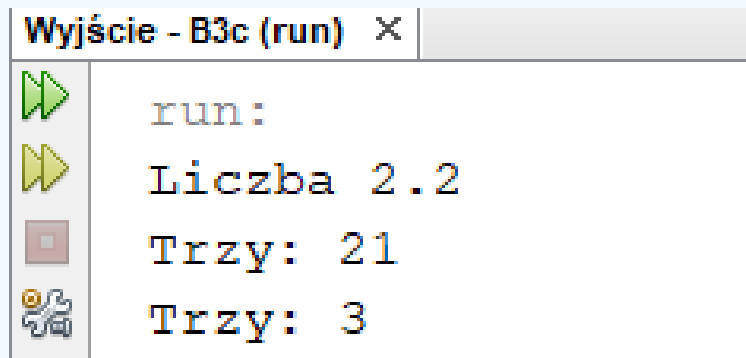
Do łączenia łańcuchów znakowych został wykorzystany przeładowany znak dodawania, który w poniższym przykładzie służy do dopisania ciągu znaków do istniejącego obiektu.

```
String s1 = "Łańcuch 1 " + "Łańcuch 2";  
String s2 = "Łańcuch znaków ";  
s2 += " i znaki dodane";
```

Konkatenacja ciągu znaków z innymi typami

W języku Java dokonywana jest automatyczna zamiana różnych typów danych do ciągu znaków. Poniższy przykład ilustruje konwersję danych.

```
double a = 2.2;  
String s = "Liczba " + a;  
System.out.println(s);  
s = "Trzy: " + 2 + 1;  
System.out.println(s);  
s = "Trzy: " + (2 + 1);  
System.out.println(s);
```



The screenshot shows a window titled "Wyjście - B3c (run) X". It contains the following output:

```
run:  
Liczba 2.2  
Trzy: 21  
Trzy: 3
```

The output demonstrates the automatic conversion of data types to strings in Java. The first line shows the concatenation of a string and a double. The second line shows the concatenation of a string and two integers, resulting in "Trzy: 21". The third line shows the concatenation of a string and a parenthesized expression, resulting in "Trzy: 3".

Konwersja do ciągu znaków i metoda `toString()`

Gdy Java konkatenuje dane do reprezentacji tekstowej wywołuje jedną z przeciążonych wersji metody *valueOf()* zdefiniowanej w klasie *String*. Metoda ta jest przeciążona dla wszystkich typów prostych oraz dla typu *Object*. Dla obiektów metoda *valueOf()* wywołuje metodę *toString()* dla przekazanego obiektu.

Każda klasa implementuje metodę *toString()*, ponieważ jest ona zdefiniowana w klasie *Object*. Domyślna implementacja nie jest na ogół najlepsza i z tego powodu w tworzonych klasach przysłania się oryginalną wersję własną metodą *toString()*. Ogólna postać metody *toString()* jest następująca:

String toString()

```
package b3d;
class Punkt{
    double x, y;
    Punkt(double x, double y){
        this.x = x;
        this.y = y;
    }
    public String toString() {
        return "(" + this.x + "; " + this.y + ")";
    }
}

public class B3d {
    public static void main(String[] args) {
        Punkt p = new Punkt(1.2, -2.7);
        System.out.println("Punkt p: " + p);
    }
}
```

run:

Punkt p: (1.2; -2.7)

Metoda *charAt()* i *getChars()*

Do pobrania pojedynczego znaku z obiektu `String` służy metoda *charAt()*. Do wydobywania większej liczby znaków, używamy metody *getChars()*. Składnia obu metod jest następująca:

char *charAt*(**int** *where*)

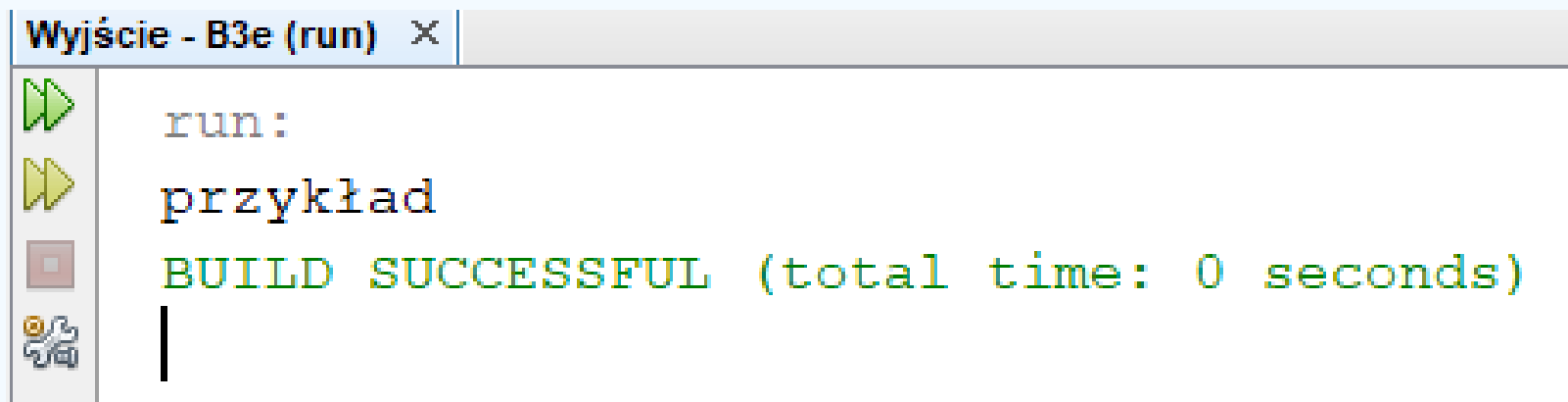
void *getChars*

(**int** *sourceStart*, **int** *sourceEnd*, **char** *target[]*, **int** *targetStart*)

Element *sourceStart* określa indeks początku podciągu, natomiast *sourceEnd* określa indeks o jeden mniejszy niż koniec podciągu. Zatem podciąg zawiera znaki o indeksach od *sourceStart* do *sourceEnd*-1. Element *target* określa tablicę , do której zostaną wpisane znaki. Wpisywanie rozpoczyna się od indeksu wskazanego przez *targetStart*. Tablica powinna posiadać taki wymiar aby mogła pomieścić cały pobrany podciąg.

Przykład użycia metody *getChars()*

```
String s = "To jest przykład użycia metody getChars()";  
int start = 8;  
int end = 16;  
char buf[] = new char[end - start];  
  
s.getChars(start, end, buf, 0);  
System.out.println(buf);
```



Metoda *getBytes()* i *toCharArray()*

Alternatywą dla metody *getChars()* jest metoda *getBytes()*, która zwraca tablicę bajtów. Tablica ta zawiera znaki obiektu *String* po dokonaniu domyślnej konwersji ze znaków na bajty dla danej platformy.

byte[] *getBytes()*

Jeżeli chcemy przenieść wszystkie znaki obiektu *String* do tablicy znaków, najczęściej używamy metody *toCharArray()*. Ogólna postać tej metody jest następująca:

char[] *toCharArray()*

Metoda ta jest bardzo często używana przez programistów w celu ułatwienia sobie życia.

Porównywanie łańcuchów znaków

Do sprawdzania równości dwóch łańcuchów znaków służy metoda *equals()*. Jej ogólna postać jest następująca.

boolean equals(**Object** *str*)

Element *str* to obiekt `String` porównywany z obiektem, dla którego została wywołana metoda. Metoda *equals()* zwraca wartość *true*, jeśli oba ciągi zawierają identyczne znaki w tej samej kolejności. W przeciwnym wypadku zwracana jest wartość *false*. Przy porównywaniu ma znaczenie wielkość liter.

Do przeprowadzenia porównywania bez uwzględniania wielkości liter służy metoda *equalsIgnoreCase()*.

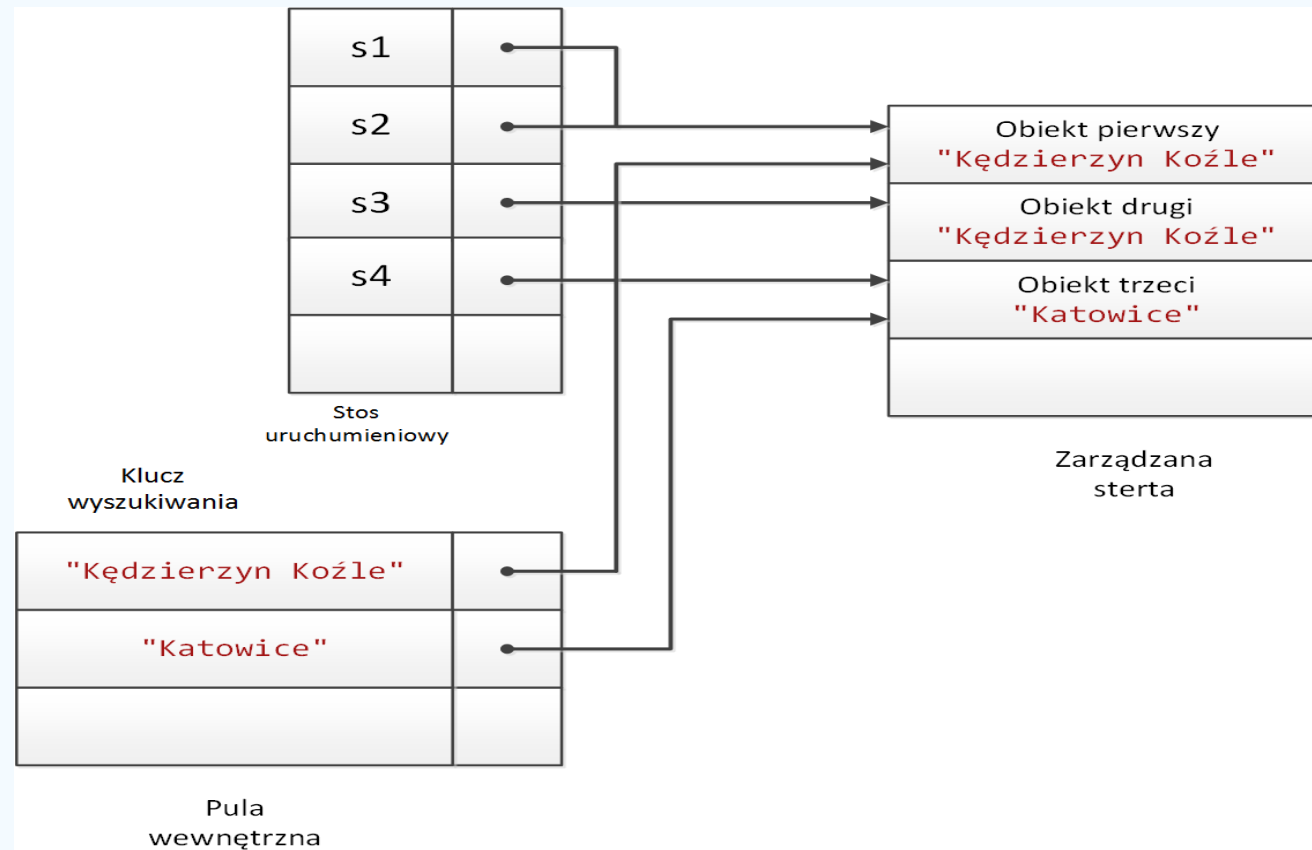
boolean equalsIgnoreCase(**Object** *str*)

Metoda *equals()* a operator ==

Należy zaznaczyć, że metoda `equals()` i operator `(==)` działają inaczej. Metoda porównuje znaki znajdujące się w obiekcie. Operator `==` porównuje referencje do obiektów, by sprawdzić, czy wskazują na ten sam egzemplarz. Prosty przykład ilustruje sposób porównania obiektów `String`.

```
String s1 = "Kędzierzyn Koźle";  
String s2 = "Kędzierzyn Koźle";  
String s3 = new String("Kędzierzyn Koźle");  
String s4 = "Katowice";  
System.out.println(s1==s2);    // true  
System.out.println(s1==s3);    // false  
System.out.println(s1.equals(s3));    // true  
System.out.println(s1=="Kędzierzyn Koźle");    //true  
System.out.println(s1.equals(s4));    // false
```

Podczas kompilacji pierwszy łańcuch zostaje umieszczony w puli wewnętrznej, jego referencje na zarządzanej sterce. Kiedy kompilator napotka na drugi taki sam łańcuch, który jest już umieszczony w puli wewnętrznej, środowisko uruchomieniowe przypisze obiekt s2 do tego samego łańcucha co s1. W dalszej kolejności konstruktor new utworzy nowy łańcuch na zarządzanej sterce. Po napotkaniu na nowy łańcuch pula wewnętrzna zostanie powiększona o nowy obiekt związany z łańcuchem s4.



Metoda *compareTo()*

Do porównywania łańcuchów znaków używamy metody *compareTo()*. Postać metody jest następująca.

int compareTo(**String** *str*)

Element *str* to obiekt dla którego została wywołana metoda. Wynik zwrócony przez metodę zależy od wyniku porównania łańcucha wywołanego do łańcucha porównywanego i jest opisany w tabeli poniżej.

Wartość	Znaczenie
Mniejsza od zera	Wywołany obiekt zawiera ciąg mniejszy od <i>str</i>
Większa od zera	Wywołany obiekt zawiera ciąg większy od <i>str</i>
Równa zero	Oba ciągi są takie same

Metoda *compareTo()*

Do porównywania łańcuchów znaków bez uwzględnienia wielkości liter służy metoda `compareTo()` o następującej składni.

int `compareTo( String str)`

```
String s1 = "Java";  
String s2 = "java";  
System.out.println(s1.compareTo(s2));    // -32  
System.out.println(s2.compareTo(s1));    // 32  
System.out.println(s1.compareToIgnoreCase(s2));    // 0
```

Dużą litera "M" znajduje się przed małą literą "m", dlatego ciąg znaków `st1` jest mniejszy od ciągu znaków `st2`. W trzeciej instrukcji ciągi znaków są porównywane bez uwzględnienia różnicy wielkości liter.

Porównywanie polskich znaków

Niestety metoda *compareTo()* nie porównuje prawidłowo polskich liter zgodnie z alfabetem polskim. Do tego celu należy zaprojektować własną metodę sprawdzającą łańcuchy znaków zgodnie ze słownikiem polskim.

```
static int compare(String arg1,String arg2)
{
    Locale locale =new Locale("pl","PL");
    // Utworzenie obiektu lokalnych ustawień
    Collator col = Collator.getInstance(locale);
    // Utworzenie komparatora instancji polskiej.
    col.setStrength(Collator.PRIMARY);
    // Wpisanie sposobu porównywania znaków Unicode.
    return col.compare(arg1, arg2);
    // Zwrócenie wyniku porównania.
}
```

```
public static void main(String[] args) {  
    String s1 = "a";  
    String s2 = "a";  
    String s3 = "b";  
    System.out.println(compare(s1,s2)); // -1  
    System.out.println(compare(s2,s3)); // -1  
}
```

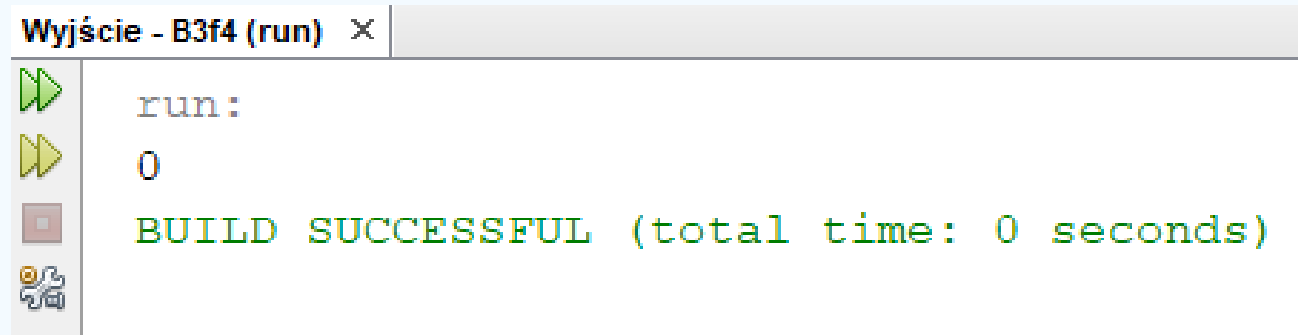
Analogicznie możemy zaprojektować metodę porównującą łańcuchy znaków bez uwzględniania wielkości liter. Do tego celu została wykorzystana zamieniająca wszystkie litery łańcucha na duże. Umożliwia to porównanie łańcuchów bez uwzględniania wielkości liter.

Funkcja *compareToIgnoreCase()*

```
static int compareToIgnoreCase(String arg1,String arg2)
{
    Locale locale =new Locale("pl","PL");
    // Utworzenie obiektu lokalnych ustawień
    Collator col = Collator.getInstance(locale);
    // Utworzenie komparatora instancji polskiej.
    col.setStrength(Collator.PRIMARY);
    // Wpisanie sposobu porównywania znaków Unicode.
    return col.compare(arg1.toUpperCase(locale),
                       arg2.toUpperCase(locale));
    // Zwrócenie wyniku porównania.
}
```

Sprawdzenie działania programu

```
public static void main(String[] args) {  
    String s1 = "kędzierzyn koźle";  
    String s2 = "KĘDZIERZYN KOŹLE";  
    System.out.println(compareIgnoreCase(s1,s2)); // 0  
}
```



The screenshot shows a terminal window titled "Wyjście - B3f4 (run)". It contains the following text: "run:" followed by "0" on the next line, and "BUILD SUCCESSFUL (total time: 0 seconds)" on the third line. On the left side of the terminal, there are four icons: a green double arrow, a yellow double arrow, a red square, and a bug icon.

```
Wyjście - B3f4 (run) ×  
run:  
0  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Formatowanie łańcuchów znaków

Do formatowania łańcuchów znakowych służy metoda *format()* z klasy `String`. Dostępne są dwie wersje metody. Pierwsza wykorzystuje przy formatowaniu domyślną lokalizację. Druga wersja umożliwia wpisanie określonej lokalizacji regionalnej i sformatowanie łańcucha w zadany sposób. Składnia metod przedstawiona jest poniżej.

String *format(String format, Object... args)*

String *format(Locale l, String format, Object... args)*

Łańcuch formatujący

Łańcuch formatujący zawiera znaki dwojakiemu rodzaju: zwykłe znaki, które są bezpośrednio kopiowane do strumienia i specyfikacje przekształcenia, które wskazują sposób przekształcenia i wypisanie argumentów funkcji. Specyfikacja przekształcenia rozpoczyna się znakiem % i kończy znakiem przekształcenia. Między znakiem przekształcenia mogą występować (w podanej kolejności):

- Modyfikatory (w dowolnej kolejności, które wpływają na postać wyniku:
 - dosunięcie przekształcanego argumentu do lewego końca jego pola;
 - + wypisanie liczby zawsze ze znakiem;

odstęp poprzedzenie wyniku znakiem ostępu, jeśli jego pierwszym znakiem nie jest plus albo minus;

0 dla przekształceń liczbowych: uzupełnienie liczby wiodącymi zerami do pełnego rozmiaru pola;

alternatywna postać wyniku: dla formatu x lub X wynik różny od zera poprzedza się 0x lub 0X; dla formatu e, E, f, g i G wypisana liczba zawsze będzie miała kropkę dziesiętną (dla g i G nieznaczące zera zostaną usunięte).

- Liczba określająca minimalny rozmiar pola. Przekształcony argument zostanie wypisany w polu o rozmiarze nie mniejszym od wskazanego (jeśli to konieczne większym).

- Jeśli przekształcany argument jest krótszy od rozmiaru pola, to zostanie dopełniony z lewej strony lub z prawej , jeśli zlecono dosunięcie w lewo) do pełnego rozmiaru pola. Znakiem wypełniającym jest zwykle odstęp, chyba że polecono dopisywanie zerami – wówczas jest nim znak 0.
- Kropka oddzielająca rozmiar pola od precyzji.
- Liczba (precyzja) określająca: maksymalną liczbę wypisywanych znaków tekstu; liczbę cyfr wypisywanych po kropce dziesiętnej w specyfikacji e, E lub f; Liczbę cyfr znaczących w specyfikacjach g lub G; minimalną liczbę cyfr wypisywanych dla argumentu całkowitego (żądany rozmiar pola osiąga się przez dopisanie wiodących zer).

- Modyfikator długości h, l (litera l) lub L. Litera h wskazuje, że odpowiedni argument jest typu short lub unsigned short; litera l wskazuje, że argument jest typu long; litera L wskazuje że argument jest typu long double.

Zamiast liczb określających rozmiar pola lub precyzję można podać znak *. W tym przypadku żadną wartość oblicza się podstawie następnego argumentu (lub następnych argumentów) funkcji. Argument ten musi być typu int.

Jeśli znaki następujące po znaku % nie są znakami przekształcenia, to skutek takiej specyfikacji nie jest określony.

Znak	Typ argumentu	Przekształcenie do postaci
d, i	int	liczby dziesiętnej ze znakiem
x, X	int	liczby szesnastkowej bez znaku
u	int	liczby dziesiętnej bez znaku
c	int	pojedynczego znaku
s	String	znaki tekstu
f	double	liczby dziesiętnej <i>$[-]mmm.ddd$</i>
e, E	double	naukowej <i>$[-]m.ddde\pm xx$</i>

Znak	Typ argumentu	Przekształcenie do postaci
g, G	double	możliwie najkrótszej postaci
n	int[]	liczba dotychczas wypisanych znaków w tym wywołaniu funkcji.
%		nie ma żadnego przekształcenia argumentu – wypisanie znaku %.
t	Data/Czas	konwersja daty i czasu
tA	Data/Czas	pełna nazwa dnia tygodnia
tB	Data/Czas	pełna nazwa dnia miesiąca

Metoda *format()* przykłady

```
double a = 78.89;  
String sz3 = String.format("a = %6.2f", a);  
System.out.println(sz3);  
String name = "Jan Woźnica";  
int lat = 24;  
String sz4 = String.format  
    ("Nazywam się %s i mam %d lata", name, lat);  
System.out.println(sz4);
```



run:



a = 78,89



Nazywam się Jan Woźnica i mam 24 lata



BUILD SUCCESSFUL (total time: 0 seconds)

Formatowanie daty i czasu przykłady

```
System.out.printf("now: %s%n", LocalDateTime.now());  
System.out.printf("Apr 15, 1994 @ 11:30am: %s%n",  
    LocalDateTime.of(1994, Month.APRIL, 15, 11, 30));  
System.out.printf("now (from Instant): %s%n",  
    LocalDateTime.ofInstant(Instant.now(),  
        ZoneId.systemDefault()));  
System.out.printf("6 months from now: %s%n",  
    LocalDateTime.now().plusMonths(6));  
System.out.printf("6 months ago: %s%n",  
    LocalDateTime.now().minusMonths(6));
```

```
now: 2018-06-25T15:21:52.911
```

```
Apr 15, 1994 @ 11:30am: 1994-04-15T11:30
```

```
now (from Instant): 2018-06-25T15:21:52.927
```

```
6 months from now: 2018-12-25T15:21:52.927|
```

```
6 months ago: 2017-12-25T15:21:52.927
```

Niestandardowy format liczb

Środowisko formatowania obsługuje specjalne znaki sortowania umożliwiające konstruowanie niestandardowych formatów liczb. Do najczęściej stosowanych znaków należą (#), (0), (,) (.), (%) i (;). W przykładzie poniżej podano kilka niestandardowych sposobów formatowania liczby:

```
String p1 = "###,###.##";  
DecimalFormat d1 = new DecimalFormat(p1);  
String f1 = d1.format(123456789.129);  
System.out.println(f1);  
String p2 = "000,000.###";  
DecimalFormat d2 = new DecimalFormat(p2);  
String f2 = d2.format(1234.123);  
System.out.println(f2);
```

123	456	789,13
001	234,	123

Konwersja danych – metoda *valueOf()*

Metoda *valueOf()* konwertuje dane z ich wewnętrznego formatu na postać znakową. Metoda jest przeciążona dla typu *Object*, klasy bazowej wszystkich klas w języku Java. Niektóre wersje tej metody:

static String *valueOf*(**double** *num*)

static String *valueOf*(**long** *num*)

static String *valueOf*(**Object** *num*)

static String *valueOf*(**char** *chars*[])

Przekazanie obiektu do metody *valueOf()* spowoduje, że metoda wywoła ona metodę *toString()* obiektu. Dla większości tablic metoda *valueOf()* zwraca informację o tym, że przekazany typ jest tablicą. Wyjątek stanowi tablica *char* – wtedy metoda zwraca obiekt *String* zawierający znaki z przekazanej tablicy. Ogólna postać metody ma postać:

static String *valueOf*(**char** *chars*[], **int** *startIndex*, **int** *numChars*)

Zamiana wielkości liter

W celu wprowadzenia zmiany wielkości liter występujących w ciągu znaków możemy zastosować metody:

String toLowerCase()

String toLowerCase(**Locale** *locale*)

String toUpperCase()

String toUpperCase(**Locale** *locale*)

```
String s3 = "To jest długi, długi, długi łańcuch znaków";  
System.out.println(s3.toLowerCase());  
System.out.println(s3.toUpperCase());
```

run:

```
to jest długi, długi, długi łańcuch znaków  
TO JEST DŁUGI, DŁUGI, DŁUGI ŁAŃCUCH ZNAKÓW
```

Usuwanie białych znaków

W celu usunięcia białych znaków z początku i końca ciągu znaków można wykorzystać metodę *trim()* postaci:

String trim()

```
String s4 = "    Komputer    ".trim();  
System.out.println(s4);
```

Metoda trim() jest bardzo użyteczna , gdy przetwarzane są polecenia przekazywane przez użytkownika. Wtedy zachodzi konieczność usunięcia niepotrzebnych białych znaków i pobrania rdzenia informacji.

Dzielenie ciągu znaków

Do dzielenia ciągu znaków wykorzystywana jest metoda *split()*. Metoda *split()* dokonuje podzielenia łańcucha znaków pomiędzy dopasowaniem zadany za pomocą wyrażenia regularnego. Metoda ma postać:

String[] split(**String** *regx*)

String[] split(**string** *regx*, **int** *limit*)

```
String s1 = "1,2|3,4|5,6|7,8|9,10";  
String [] tablica = s1.split("[|]");  
for(String s: tablica)  
    System.out.println(s);
```

```
run:|  
1,2  
3,4  
5,6  
7,8  
9,10
```

Wczytanie z linii poleceń dwóch liczb całkowitych przedzielonych spacjami

```
BufferedReader in =  
    new BufferedReader(new InputStreamReader(System.in));  
System.out.println  
    ("Wpisz dwie liczby całkowite przedzielone spacjami");  
String st = in.readLine();  
// Przeczytanie łańcucha znaków  
    st = st.trim();  
    String [] ds = st.split("\\s");  
// Podzielenie łańcucha znaków na dwie liczby  
    int i1 = Integer.parseInt(ds[0]);  
// Wczytanie pierwszej liczby  
    int i2 = Integer.parseInt(ds[ds.length - 1]);  
// Wczytanie drugiej liczby  
    System.out.printf("i1=%d i2=%d\n", i1, i2);  
// Wypisanie przeczytanych liczb
```

Działanie programu

```
Wyjście - B3h1 (run) X
run:
Wpisz dwie liczby całkowite przedzielone spacjami
1 2
i1=1 i2=2
BUILD SUCCESSFUL (total time: 5 seconds)
```

Metodę *Split()* możemy użyć do pobrania liczb z ciągu znakowego. W celu zabezpieczenia się przed wpisaniem wielu spacji pomiędzy liczbami należy sprawdzić, czy pobrany ciąg znaków nie jest pusty.

```
package b3h2;
import java.text.*;
import java.util.*;
public class B3h2 {
    public static void main(String[] args)
        throws ParseException {
        Locale locale = new Locale("pl", "PL");
        NumberFormat pl =
            NumberFormat.getInstance(locale);
        String st = "1,2      3,4 5,6 7,8 9,1";
        String [] tablica = st.split("\\s");
        double w = 0.0;
```

```
for(String s: tablica)
if (!s.isEmpty())
{
    double d = pl.parse(s).doubleValue();
    System.out.printf("%g%n", d);
    w += d;
}
System.out.printf("Suma=%g%n", w);
}
```

UWAGA: Należy zauważyć, że *parse()* nigdy nie zwraca wartości *null*, co może powodować wyrzucenie wyjątku *NullPointerException*. Dlatego trzeba usunąć z programu sprawdzanie błędu *ParseException*.

```
run:
1,20000
3,40000
5,60000
7,80000
9,10000
Suma=27,1000
```

Metoda *concat()*

Do łączenia dwóch tekstów służy metoda *concat()* o następującej postaci:

String concat(**String** *str*)

```
String s1 = "Łańcuch pierwszy + ";  
String s2 = "łańcuch drugi";  
String s3 = s1.concat(s2);  
System.out.println(s3);
```

run:

```
Łańcuch pierwszy + łańcuch drugi  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Metda substring()

Do pobrania podciągu łańcucha znaków służy metoda `substring()` postaci:

String `substring( int startIndex )`

String `substring( int startIndex, int endIndex )`

```
String s4 = "To jest przykład programu";  
String s5 = s4.substring(8, 16);  
System.out.println(s5);
```

run:

przykład

BUILD SUCCESSFUL (total time: 0 seconds)

Metoda *regionMatches()*

Metoda *regionMatches()* porównuje konkretny obszar jednego ciągu znaków z konkretnym obszarem drugiego ciągu znaków. Postacie ogólne obu wersji:

```
boolean regionMatches( int startIndex, String str2,  
                        int str2StartIndex, int numChars)
```

```
boolean regionMatches( boolean ignoreCase, int startIndex,
                        String str2, int str2StartIndex, int numChars)
```

```
boolean b = "Java".regionMatches(true, 0, "java", 0, 4);
System.out.println(b);
```

time

Metoda *startsWith()* i *endsWith()*

Metoda `startsWith()` sprawdza, czy przekazany ciąg znaków znajduje się na początku łańcucha. Analogicznie metoda `endsWith()` sprawdza, czy przekazany ciąg znaków znajduje się na końcu łańcucha.

boolean `startsWith( String str)`

boolean `endsWith( String str)`

```
boolean b1 = "Kowalski".startsWith("K");  
boolean b2 = "Kowalski".endsWith("ski");  
System.out.println(b1 + " " + b2);
```

```
run:
```

```
true true
```

Metody podmiany części łańcucha znaków

Istnieje kilka wariantów metody *replace()*. Poniżej przedstawiono niektóre dostępne wersje.

`replace ( char oldChar, char newChar )`

Zwraca nowy łańcuch powstały z przekazanego łańcucha, w którym zamieniono wszystkie wystąpienia *oldChar* na *newChar*.

`replaceAll( String regex, String replacment )`

Zamień każdy podciąg łańcucha, który jest zgodny z wyrażeniem regularnym na zadany łańcuch znaków.

`replaceFirst( String regex, String replacment )`

Zamień pierwszy podciąg łańcucha, który jest zgodny z wyrażeniem regularnym na zadany łańcuch znaków.

Przykłady podmiany części łańcucha znaków

```
System.out.println(s8);  
String s9 = "To jest długi, długi, długi łańcuch znaków";  
s9 = s9.replaceAll("długi", "krótki");  
System.out.println(s9);  
s9 = s9.replaceFirst("krótki", "długi");  
System.out.println(s9);
```

run:

Agxtx

To jest krótki, krótki, krótki łańcuch znaków

To jest długi, krótki, krótki łańcuch znaków

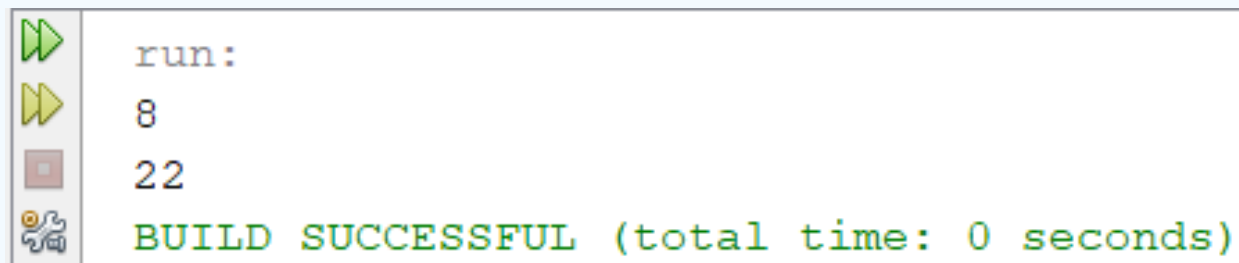
Wyszukiwanie podciągów znaków w łańcuchu

Czasami jest konieczne wyszukiwanie w ciągu znaków określonego zestawu znaków. W tym celu można wykorzystać kilka różnych metod. Wyszukanie indeksu znaku lub ciągu znaków w łańcuchu możemy dokonać przy użyciu metod *indexOf()* lub *lastIndexOf()*. Metoda *indexOf()* zwraca pozycję pierwszego wystąpienia określonego słowa w ciągu znaków, natomiast metoda *lastIndexOf()* zwraca pozycję ostatniego wystąpienia tego słowa. W przypadku niepowodzenia zwracana jest wartość -1. Składnia metod:

```
int indexOf( String str)
```

```
int lastIndexOf( String str)
```

```
String st =  
    "To jest długi, długi, długi ciąg znaków...";  
System.out.println(st.indexOf("długi"));  
System.out.println(st.lastIndexOf("długi"));
```



```
run:  
8  
22  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Poniższe wersje określają miejsce, od którego rozpoczyna się przeszukanie.

int indexOf(**String** *str*, **int** *startIndex*)

int lastIndexOf(**String** *str*, **int** *startIndex*)

Wyszukiwanie wszystkich wystąpień szukanego łańcucha

```
int position = -1;  
do  
{  
    position = st.indexOf("długi", ++position);  
    if (position >= 0)  
        System.out.println(position);  
} while (position >= 0);
```

 run:
 8
 15
 22
BUILD SUCCESSFUL (total time: 0 seconds)